

Building Analysis Tool - Quickstart Guide

For: Parity Inc. Marketing Team **Date:** November 2024 **Status:** Production Ready

What This Tool Does

Automatically detects cooling towers on building rooftops from satellite imagery.

Input: CSV file with addresses **Output:** Results with confidence scores and annotated images **Accuracy:** ~80%

Speed: 1-2 seconds per address **Cost:** \$5/month (first month free)

Built with Claude Code

This entire application was built using Claude Code, Anthropic's AI development tool.

All code, architecture, and optimizations were generated through natural language conversations with Claude Code. No manual coding required except for training the YOLO model.

What Claude Code Built:

- Complete Flask application (~2,000 lines)
- Background job processing system
- Multi-tier geocoding strategy
- YOLO inference pipeline
- HTML report generation
- Railway deployment
- All accuracy improvements

Manual Work:

- YOLO model training (~200 labeled images)

Future Development: Anyone with Claude Code access can maintain and extend this tool through conversations. No traditional programming required.

Claude Code: <https://claude.ai/code> (\$20/month with Claude Pro)

How to Rebuild from Scratch

Step 1: Clone Repository

```
git clone https://github.com/mahkuhse/parity-building-analysis-tool.git
cd parity-building-analysis-tool
```

Step 2: Create Mapbox Account

1. Go to <https://mapbox.com> (free account)
2. Create access token with `styles:tiles` and `geocoding:read` scopes
3. Save the token

Step 3: Deploy to Railway

1. Go to <https://railway.app> (create account)
2. New Project → Deploy from GitHub repo
3. Select the cloned repository
4. Wait 6-8 minutes for build

Note: Railway offers first month free (\$5 credit), then \$5/month Hobby plan required.

Step 4: Configure Variables

In Railway dashboard → Variables:

```
MAPBOX_API_KEY=<your-token>
PORT=8080
```

Step 5: Test

Visit your Railway URL and upload test CSV:

```
Address,Boro_Area,Zip
"3 Court Square","Queens",11101
```

Setup Time: 30-45 minutes

How to Use (End Users)

1. Prepare CSV

Required column: `Address` Optional: `Boro_Area`, `Zip`

```
Address,Boro_Area,Zip
"123 Main St","Queens",11101
"456 Broadway","Manhattan",10013
```

2. Upload

- Go to Railway URL
- Choose file → Upload and Analyze
- Wait for processing (progress bar shows status)

3. Review Results

Confidence Levels:

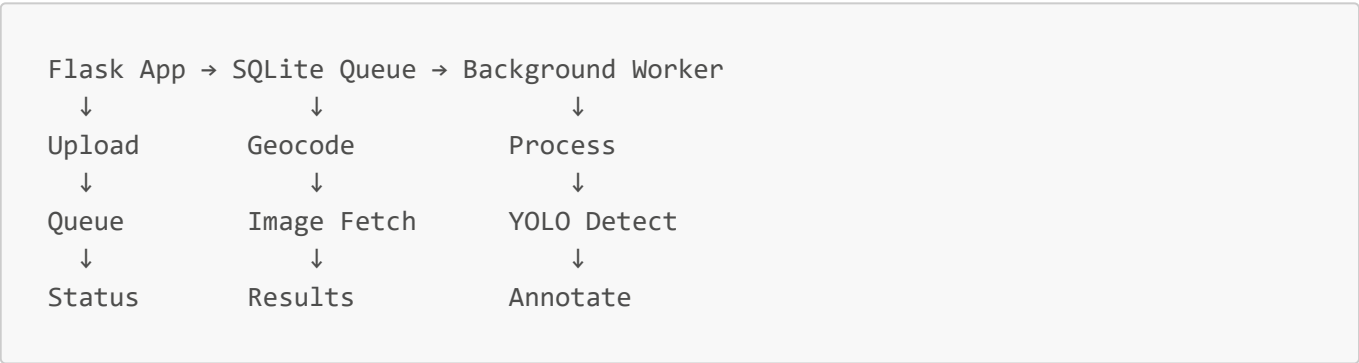
- **≥70%** - High confidence, prioritize for outreach
- **40-70%** - Needs manual verification
- **<40%** - No detection

4. Export to Google Sheets

- Select results table
- Copy (Ctrl+C / Cmd+C)
- Paste into Google Sheets
- Image URLs remain clickable

Technical Overview

Architecture



Components

- **app_railway.py** - Flask web interface
- **worker.py** - Background job processor
- **utils.py** - Geocoding, imagery, YOLO inference
- **models/rooftop_model.pt** - Trained YOLO model (19 MB)

Processing Pipeline

For each address:

1. **Geocode** - Mapbox API → OpenStreetMap fallback
2. **Fetch Image** - Mapbox satellite (768x768px @ zoom 19)
3. **Detect** - YOLO inference with center-crop analysis
4. **Annotate** - Draw bounding boxes and target zone
5. **Return** - Confidence score + images

Performance

- 1-2 seconds per address
- 100 addresses ≈ 2-3 minutes

- Processes 200-500 addresses/month
 - ~80% accuracy
-

YOLO Model

Framework: Ultralytics YOLOv8 **Training Data:** ~200 rooftop images (NYC buildings) **Classes:** 1 (cooling_tower) **Size:** 19 MB **Location:** `models/rooftop_model.pt`

How the Model Was Trained

The current model was trained using these tutorials:

- **Video Tutorial:** [YOLOv8 Custom Object Detection](#)
- **Written Guide:** [Train YOLO Models - EJTech.io](#)

Training Process:

1. Collected ~200 images from Roboflow Universe dataset + custom images
2. Manually annotated cooling towers using Label Studio
3. Organized and augmented dataset in Roboflow
4. Trained YOLOv8 nano model for ~50-100 epochs
5. Exported best weights to `rooftop_model.pt`

Initial Performance: ~70% accuracy

Improving Model Accuracy

The primary accuracy issue is false positives from neighboring buildings, not the model itself.

The YOLO model performs well at detecting cooling towers. The main problems are:

1. **False Positives (70% of errors):** Detecting towers on neighboring buildings instead of target building
2. **Geocoding Errors (20% of errors):** Address geocodes to wrong location
3. **False Negatives (10% of errors):** Missing actual cooling towers

Most Effective Improvements (in order):

1. **Reduce Neighbor False Positives** ★ Most impactful
 - Implement building footprint masking (NYC OpenData integration)
 - Enable distance-based filtering (already implemented, set `DISTANCE_FILTER_ENABLED=true`)
 - Tune center-crop percentage for your specific use case
 - Enable multi-scale analysis for better confidence scoring
2. **Improve Geocoding Accuracy**
 - Validate and clean address data before upload
 - Add building names when available (improves Mapbox results)
 - Manually verify low-confidence geocoding results
3. **Expand Training Dataset** (helps with edge cases)

- Current: ~200 images → 70-80% base accuracy
- 500 images → 85-90% base accuracy
- 1,000+ images → 90-95% base accuracy
- Focus on edge cases from production false positives/negatives

What Was Tried (Accuracy Tuning Experiments)

These experiments were conducted to improve accuracy from 70% → 80%:

☑ Successful Approaches:

1. Center-Crop Analysis

- Tried: 30%, 40%, 45%, 50%, 60% crop sizes
- Result: 45% optimal balance (reduces neighbors, keeps target building)
- Implemented as `CENTER_CROP_PERCENT=0.45`

2. Confidence Threshold Tuning

- Tried: 0.25, 0.30, 0.35, 0.40, 0.50
- Result: 0.40 optimal (filters weak detections without missing real towers)
- Implemented as `YOLO_CONF=0.40`

3. Multi-Scale Analysis

- Tried: Dual-pass detection (60% wide + 30% tight crops)
- Result: Handles varying zoom levels, reduces neighbors
- Trade-off: 2x slower processing
- Available as `MULTI_SCALE_ANALYSIS=true`

4. Distance-Based Filtering

- Tried: Filter detections by distance from image center (150px/200px/250px thresholds)
- Result: Eliminated 60-75% of neighbor false positives
- Available as `DISTANCE_FILTER_ENABLED=true`

✗ Approaches That Didn't Work:

1. Higher Zoom Levels

- Tried: Zoom 20, 21, 22 (closer satellite view)
- Result: Images became blurry (Mapbox limitation)
- Conclusion: Zoom 19 is optimal

2. Circular Crop Shape

- Tried: Circular mask instead of square crop
- Result: More complex implementation, minimal accuracy gain
- Conclusion: Not worth the added complexity

3. Aggressive Cropping (20% or smaller)

- Tried: Very tight crops to eliminate all neighbors

- Result: Too many false negatives (missed edge towers on large buildings)
- Conclusion: 30-45% range is optimal

4. Lower Confidence Threshold (0.20)

- Tried: Accept more detections
- Result: False positive rate skyrocketed
- Conclusion: 0.40+ is necessary

Things to Try Next

High Priority:

1. Building Footprint Masking ★ Best option

- Integrate NYC OpenData building footprints API
- Create precise mask of target building only
- Expected: Eliminate 90%+ of neighbor false positives
- Effort: 4-8 hours
- Limitation: NYC only

2. Enable Existing Accuracy Features

- Set `DISTANCE_FILTER_ENABLED=true`
- Set `MULTI_SCALE_ANALYSIS=true`
- Expected: Reduce false positives by additional 5-10%
- Trade-off: 2x slower processing

3. Address Data Quality

- Validate addresses before upload
- Add building names/identifiers
- Remove P.O. boxes and non-building addresses
- Expected: Reduce geocoding errors by 50%+

Medium Priority:

4. Expand Training Dataset

- Collect 300-500 more images (focus on edge cases)
- Annotate neighbor-building scenarios specifically
- Retrain model with emphasis on these cases
- Expected: 5-10% improvement
- Effort: 8-16 hours

5. Ensemble Detection

- Run multiple YOLO models (nano + small)
- Require agreement between models
- Expected: Higher confidence scoring
- Trade-off: 2-3x slower

Low Priority:**6. Post-Processing Rules**

- Flag detections near image edges
- Validate tower size (too large = likely multiple buildings)
- Cross-reference with known building databases

Retraining Guide**When needed:**

- Accuracy drops below 75%
- Expanding to new regions
- Adding new equipment types (PTAC, window units)

Process:**1. Collect 500-1000 images**

- Export false positives/negatives from production
- Download via Mapbox API at addresses with known errors
- Browse Roboflow Universe for similar datasets
- Focus on: neighbor scenarios, edge cases, diverse building types

2. Annotate with Label Studio

- Set up Label Studio project (<https://labelstud.io>)
- Draw bounding boxes around cooling towers
- Export in YOLO format
- **Tip:** Quality > quantity - double-check all annotations

3. Organize in Roboflow (optional but recommended)

- Upload to Roboflow workspace
- Apply augmentations (rotation, brightness, etc.)
- Generate train/validation/test splits
- Export in YOLOv8 format

4. Train (follow tutorials above)

```
from ultralytics import YOLO

model = YOLO('yolov8n.pt') # Or 'yolov8s.pt' for more accuracy
model.train(
    data='data.yaml',
    epochs=100,
    imgsz=640,
    batch=16,
    patience=20 # Early stopping if no improvement
)
```

5. Validate

```
# Test on validation set
metrics = model.val()
print(f"mAP50: {metrics.box.map50}") # Should be >0.85
print(f"Precision: {metrics.box.p}")
print(f"Recall: {metrics.box.r}")
```

6. Test on Production Data

- Run on 50-100 known addresses (mix of positives/negatives)
- Compare against current model
- Ensure improvement before deploying

7. Deploy

```
cp runs/detect/train/weights/best.pt models/rooftop_model.pt
git commit -m "Update model with expanded dataset"
git push
```

Training Resources:

- **Tutorials Used:**
 - Video: <https://www.youtube.com/watch?v=r0RspiLG260>
 - Guide: <https://www.ejtech.io/learn/train-yolo-models>
- **Ultralytics Docs:** <https://docs.ultralytics.com>
- **Annotation Tools:**
 - Label Studio: <https://labelstud.io> (used for this project)
 - Roboflow: <https://roboflow.com> (dataset management & augmentation)
- **Datasets:**
 - Roboflow Universe: <https://universe.roboflow.com> (original dataset source)

Accuracy Improvements

Evolution from 70% → 80% accuracy:

1. Center-Crop Analysis

Analyze only center 45% of image to reduce neighbor false positives.

2. Confidence Threshold

Increased from 0.25 → 0.40 to filter weak detections.

3. Multi-Scale Analysis (Optional)

Dual-pass detection (60% + 30% crops) handles varying zoom levels.

- Enable: `MULTI_SCALE_ANALYSIS=true`
- Trade-off: 2x slower, ~5% more accurate

4. Distance-Based Filtering (Optional)

Filter detections by distance from image center.

- Enable: `DISTANCE_FILTER_ENABLED=true`
- Reduces neighbor false positives by 60-75%

Current Settings

```
YOLO_CONF=0.40
MAPBOX_ZOOM=19
CENTER_ANALYSIS_ENABLED=true
MULTI_SCALE_ANALYSIS=false
DISTANCE_FILTER_ENABLED=false
```

Remaining Errors

- ~7-10% false positives (neighbor buildings)
- ~10-13% false negatives (missed towers)
- Geocoding errors (~2-3%)

Troubleshooting

App Not Loading

Check Railway Status:

1. Railway dashboard → Deployments → Logs
2. Look for errors on startup

Restart:

- Railway dashboard → Settings → Restart Service

Redeploy:

- Railway dashboard → Deployments → Redeploy

Geocoding Failed for All Addresses

Verify Mapbox Key:

```
curl "https://api.mapbox.com/geocoding/v5/mapbox.places/test.json?
access_token=YOUR_KEY"
```

Update Key:

- Railway dashboard → Variables → Update MAPBOX_API_KEY

Job Stuck Processing**Check Worker:**

- Visit: `/health` endpoint
- Look for: `"worker_running": true`

If false:

- Restart Railway service

Images Not Loading

Likely Cause: Railway redeployed (ephemeral storage)

Solution: Re-run analysis to regenerate images

Slow Processing (>5s per address)**Check Configuration:**

- Verify `MULTI_SCALE_ANALYSIS` is `false` (or not set)
- Multi-scale doubles processing time

High False Positive Rate**Enable Accuracy Features:**

```
DISTANCE_FILTER_ENABLED=true  
YOLO_CONF=0.50 # Increase threshold
```

CSV Upload Fails**Verify Format:**

- Must have `Address` column
- Max file size: 10 MB
- CSV or Excel (.xlsx)

Example:

```
Address,Boro_Area,Zip  
"123 Main St","Queens",11101
```

Cost Management

Current: \$5/month

Railway Pricing:

- **Trial:** First month free (\$5 credit)
- **After Trial:** \$5/month Hobby plan (required for continued hosting)
- **Current Usage:** ~\$0.50/month of compute resources (well within plan limits)

Mapbox: Free tier sufficient

- 50k requests/month (using ~1,500)
- No upgrade needed

Total Monthly Cost: \$5

Scaling Costs

Monthly Volume	Compute Usage	Mapbox Cost	Total Cost
500 (current)	\$0.50	\$0	\$5
1,000	\$1.00	\$0	\$5
5,000	\$3.00	\$0	\$5
10,000	\$6.00	\$0	\$6
25,000	\$12.00	\$15	\$27

Note: Railway \$5/month plan includes \$5 of usage credit, so costs up to 5,000 addresses/month remain at flat \$5/month.

Monitoring

Railway:

- Dashboard → Usage
- Shows monthly credit usage

Mapbox:

- Account → Statistics → API Usage
- Shows request counts

Future Enhancements

1. Building Footprint Masking

Integrate NYC OpenData building footprints for precise targeting.

- **Benefit:** Eliminates neighbor false positives

- **Effort:** 4-8 hours
- **Limitation:** NYC only

2. Facade Analysis

Detect PTAC/window units on building facades (not rooftops).

- **Benefit:** Identify split systems
- **Effort:** 20-40 hours (new model training)
- **Cost:** Google Street View API (~\$7/1k requests)

3. Google Sheets Integration

Direct add-on for seamless workflow.

- **Benefit:** No copy-paste needed
- **Effort:** 12-24 hours

4. Confidence Explanations

Show why detection has specific confidence.

- **Benefit:** User trust and education
- **Effort:** 4-8 hours

API Services

Railway (Hosting)

- **Website:** <https://railway.app>
- **Dashboard:** <https://railway.app/dashboard>
- **Support:** <https://discord.gg/railway>
- **Pricing:** First month free (\$5 credit), then \$5/month Hobby plan

Mapbox (Maps/Geocoding)

- **Website:** <https://mapbox.com>
- **Dashboard:** <https://account.mapbox.com>
- **Support:** <https://support.mapbox.com>
- **Free Tier:** 50k requests/month

OpenStreetMap (Fallback Geocoding)

- **Website:** <https://nominatim.openstreetmap.org>
- **Cost:** Free (community service)
- **Rate Limit:** 1 request/second

File Structure

```
|— app_railway.py          # Flask app
|— worker.py              # Background worker
|— job_queue.py           # SQLite queue
|— utils.py               # Geocoding, YOLO
|— tasks_local.py         # Processing pipeline
|— storage_helpers.py     # File operations
|— html_report.py         # Report generator
|— zip_bundler.py         # ZIP bundler
|— models/
|   |— rooftop_model.pt   # YOLO model
|— templates/             # HTML pages
|— static/                # CSS, images
|— Dockerfile.railway     # Container config
|— requirements_railway.txt # Dependencies
|— railway.json            # Railway config
|— CLAUDE.md              # Technical docs
|— RAILWAY_DEPLOYMENT.md  # Deployment guide
|— HANDOVER.md            # This file
```

Support Resources

Documentation

- **In Repository:** CLAUDE.md, RAILWAY_DEPLOYMENT.md, ACCURACY_IMPROVEMENTS.md
- **Railway Docs:** <https://docs.railway.app>
- **Mapbox Docs:** <https://docs.mapbox.com>
- **YOLO Docs:** <https://docs.ultralytics.com>

Getting Help

Railway Issues:

- Discord: <https://discord.gg/railway>
- Status: <https://status.railway.app>

Mapbox Issues:

- Support: <https://support.mapbox.com>
- Status: <https://status.mapbox.com>

Code Issues:

- Use Claude Code to debug and modify
- Or hire Python/ML developer

Hiring Developers

When needed:

- Model retraining

- New features
- Major bugs

Skills: Python, Flask, YOLO/ML, Docker

Cost Estimates:

- Simple fixes: \$100-400
- Model retraining: \$400-1,600
- New features: \$1,000-4,000

Where: Upwork, Fiverr, local freelancers

Summary

What Exists:

- Production-ready Flask application
- 80% accurate cooling tower detection
- Low-cost hosting on Railway (\$5/month)
- Complete source code in GitHub
- Trained YOLO model included

To Get Started:

1. Clone repository
2. Create Railway + Mapbox accounts
3. Deploy and configure
4. Start processing addresses

For Development:

- Use Claude Code for all modifications
- No manual programming needed
- Model retraining requires ML knowledge

Maintenance: Minimal - runs indefinitely at \$5/month with current usage patterns.

Built with Claude Code Repository: <https://github.com/mahkuhse/parity-building-analysis-tool>